

Automating Hyper Hoare Logic via Predicate Transformers

Project Description for a Bachelor's Thesis

David Hagen

June 23, 2025

Supervised by Thibault Dardinier and Prof. Dr. Peter Müller
Programming Methodology Group, ETH Zürich

I. CONTEXT AND MOTIVATION

Hoare Logic [1] is a rule-based calculus for formally proving properties of all individual executions of a program. It does this by establishing a *Hoare triple* of the form $\{P\} C \{Q\}$, where P is a precondition, C is a command (or program), and Q is a postcondition. This triple holds if and only if, whenever the precondition P holds before executing C , then the postcondition Q holds afterwards (if C terminates). However, many important properties one wants to prove about programs, such as determinism or non-interference [2], cannot be expressed as properties of individual executions – they are so-called *hyperproperties* [3].

Hyperproperties describe relationships between sets of execution traces, rather than only talking about individual traces. There have been multiple approaches to extend Hoare Logic such that these hyperproperties can be supported. Several extensions – such as RHLE [4] and BiKAT [5] – push towards this goal, but the frameworks are limited, and some types of hyperproperties still cannot be expressed [6]. In contrast, *Hyper Hoare Logic (HHL)* [6] is a powerful novel Hoare logic and is expressive enough to *prove or disprove any program hyperproperty*, including $\exists^*\forall^*$ -properties (such as the violation of *generalized non-interference* [7], an important security property), for an unlimited number of executions.

To do so, HHL lifts pre- and postconditions from simple assertions to *hyper-assertions* over sets of execution states to get *hyper-triples* of the form $\{\hat{P}\} C \{\hat{Q}\}$, where $\hat{P}$ and $\hat{Q}$ are hyper-assertions. Such a hyper-triple is valid if and only if for any set of initial states S that satisfies $\hat{P}$, the set of all final states that can be reached by executing C in some state from S satisfies $\hat{Q}$.

In order to automate the verification of hyper-triples, Dardinier et al. present *Hypra* [8], the first deductive verifier for hyperproperties with arbitrary quantifier alternations. Hypra works by encoding hyperproperties and the corresponding proof rules into an intermediate verification language, namely Viper [9]. This is done by explicitly

```
1 method simple(x: Int, y: Int) returns (z: Int)
2   requires forall <_s1> :: (exists <_s2> ::
3     ↪ _s2[x] > _s1[x])
4   requires forall <_s1>, <_s2> :: _s1[y] ==
5     ↪ _s2[y]
6   ensures forall <_s1> :: (exists <_s2> ::
7     ↪ _s2[z] > _s1[z])
8 {
9   z := x + y
10 }
```

Fig. 1. A simple code example from Hypra [8]

modeling sets of program states in Viper's first-order states by computing both a lower and an upper bound for these sets of states as an approximation.

This *semantic* approach described above is currently implemented in Hypra, but it has certain limitations – most notably performance. Even for relatively small programs, the Viper encodings can become quite large and bulky. For example, consider the method `simple` in Figure 1. This seven-line Hypra snippet is compiled into a Viper program of over 100 lines, which is then again verified using Viper's backend verifier. This bulky translation imposes a significant burden on the solver and scales poorly as program size increases.

II. SYNTACTIC AUTOMATION APPROACH

To address the scalability and performance limitations of Hypra's current semantic verification model, our goal is to implement a syntactic approach as an alternative. The idea is to use and generalize the syntactic rules of Hyper Hoare Logic [6] to compute a *weakest precondition (WP)* from the postcondition. Given the fact that postconditions can relate different executions in HHL, the weakest precondition must account for every control-flow path that any of those executions might follow. Only then can we guarantee that the postcondition will hold no matter which path each execution actually takes.

```

1 if (c) {
2   x := a
3 } else {
4   x := b
5 }

```

Fig. 2. Simple if-else code snippet

A. Characterizer

To do so, our idea is to first compute a *characterizer* that characterizes all paths of a program. Usually, the weakest precondition is defined compositionally, so one might wonder why we are not computing it “directly”.

The issue arises when we encounter conditional statements in our program. In classic Hoare Logic, we have the smooth definition [10]

$$\text{WP}(\text{if } (b) \{ C_1 \} \text{ else } \{ C_2 \}, Q) = (b \wedge \text{WP}(C_1, Q)) \vee (\neg b \wedge \text{WP}(C_2, Q)),$$

where the function WP maps a command C and a postcondition Q to the weakest precondition. This disjunction works because the postcondition Q talks about *one* final state at a time.

However, in our HHL setting, a postcondition $\hat{Q}$ often relates *multiple* final states, e.g., it might compare a state that reached the then-branch with a state that reached the else-branch. No purely syntactic combination of $\text{WP}(C_1, \hat{Q})$ and $\text{WP}(C_2, \hat{Q})$ can capture such cross-path comparisons.

Our approach to overcome this issue is to enumerate all program paths first and compute a *characterizer*. A characterizer of a program C is defined as a list of pairs (pc, θ) , where each element of the list covers one path of the program.¹ The characterizer itself then covers exactly all paths of the program.

- The path condition pc is a program boolean expression, which holds on an initial state if and only if this initial state can take this path.
- The substitution θ maps each variable x of the program to an expression p , meaning that the variable x in the final state will have the value of the expression p in the initial state. Formally, the following should hold:

$$\langle C, \sigma \rangle \rightarrow \sigma' \implies \llbracket pc \rrbracket(\sigma) \implies \sigma'(x) = \llbracket p \rrbracket(\sigma),$$

where the notation $\llbracket e \rrbracket(\sigma)$ denotes the semantic evaluation of the expression e in state σ .

Example. As an illustration, consider the simple if-else code shown in Figure 2. The corresponding characterizer is:

$$[(c, x \mapsto a), (\neg c, x \mapsto b)]$$

¹In the first version, the characterizer is simplified by ignoring non-deterministic assignments. Support for such assignments will be added as part of Core Goal 2.

That is, we have two paths, one that is being explored if c holds, in which x gets updated to a , and the other one is taken if $\neg c$ holds, where x gets updated to b .

B. Obtaining the Weakest Precondition

The next step is to compute the weakest precondition² for a given loop-free program C and a given postcondition Q . We first compute the program’s characterizer as described above and then instantiate the postcondition by applying the corresponding substitution given by the characterizer for each path.

Finally, we combine the results of all paths to get the weakest precondition: For a universally-quantified state, we do a conjunction for all paths, and we *assume* the path condition. To prove the hyper-triple $\{\hat{P}\} C \{\hat{Q}\}$, we want to first obtain the weakest precondition $\hat{P}_w$ starting from the postcondition $\hat{Q}$:

$$\forall \langle \sigma \rangle. \hat{Q} \rightsquigarrow \forall \langle \sigma \rangle. (\llbracket b_1 \rrbracket(\sigma) \implies \hat{Q}_1) \wedge \dots \wedge (\llbracket b_n \rrbracket(\sigma) \implies \hat{Q}_n),$$

where for each path i :

- $\hat{Q}_i$ stands for the postcondition $\hat{Q}$ syntactically transformed under the substitution of path i with initial state σ . This transformation is applied recursively.
- b_i is the corresponding path condition.

Dually, for an existentially-quantified state, we do a disjunction for all paths, and we *assert* the path condition to get the WP:

$$\exists \langle \sigma \rangle. \hat{Q} \rightsquigarrow \exists \langle \sigma \rangle. (\llbracket b_1 \rrbracket(\sigma) \wedge \hat{Q}_1) \vee \dots \vee (\llbracket b_n \rrbracket(\sigma) \wedge \hat{Q}_n)$$

Example. Consider again the code snippet in Figure 2 and the following postcondition, asserting the existence of a maximum:

$$\exists \langle \sigma \rangle. \forall \langle \sigma' \rangle. \sigma(x) \geq \sigma'(x)$$

The weakest precondition that should be computed in this case is:

$$\begin{aligned} \exists \langle \sigma \rangle. & (\sigma(c) \wedge (\forall \langle \sigma' \rangle. (\sigma'(c) \implies \sigma(a) \geq \sigma'(a)) \wedge \\ & (\neg \sigma'(c) \implies \sigma(a) \geq \sigma'(b)))) \\ \vee & (\neg \sigma(c) \wedge (\forall \langle \sigma' \rangle. (\sigma'(c) \implies \sigma(b) \geq \sigma'(a)) \wedge \\ & (\neg \sigma'(c) \implies \sigma(b) \geq \sigma'(b)))) \end{aligned}$$

With this approach, we obtain the weakest precondition $\hat{P}_w$ for loop-free programs. Using an SMT solver (in our case we will be using Z3 [11]) we can then check whether the given precondition $\hat{P}$ implies $\hat{P}_w$, thereby establishing the validity of the hyper-triple $\{\hat{P}\} C \{\hat{Q}\}$.

C. Extending the Methodology to Loops

To handle programs with loops, we extend our approach by structurally decomposing the program at each loop occurrence. Specifically, for each loop, we divide the program into multiple components: the prefix before the loop, the loop body, and the suffix after the loop. Each part is

²The weakest precondition is typically not identical to the precondition the user aims to prove.

analyzed separately using the existing framework for loop-free code. For the loop itself, we invoke the appropriate loop rule as defined in the HHL paper [6], which controls how the individual segments can be composed again.

By applying this procedure recursively at every loop occurrence, we cover all nested and sequential loops of the program uniformly.

III. GOALS OF THE THESIS

A. Core Goals

- 1) **Implement the computation of the weakest precondition for loop-free deterministic statements.**

The primary objective of the thesis is to realize the weakest precondition approach introduced in Section II. The implementation focuses on a function that takes a loop-free statement and a postcondition (hyper-assertion) as input, and returns the corresponding weakest precondition.

- 2) **Extend the implementation to support loops, non-deterministic assignments, and errors**

The next phase involves enhancing the syntactic automation by extending its support to handle loops, non-deterministic *havoc* statements, and errors, such that it is applicable to a wider range of programs. In particular, supporting *havoc* statements requires us to extend the current notion of a characterizer.

- 3) **Evaluate the performance of the approach.**

We want to compare the execution time of the syntactic approach against the current Hypra verification pipeline. The evaluation should demonstrate whether the new method offers a more efficient alternative to the current strategy, while maintaining the soundness of the verification.

B. Extension Goals

- 1) **Explore CVC5 as an alternative SMT solver.** Investigate the use of CVC5 as an alternative to Z3 as the backend solver, focusing on compatibility and performance.

- 2) **Compare different SMT encodings.**

Compare different strategies for encoding verification conditions into SMT logic. This involves exploring various representations and evaluating their impact on solver performance.

- 3) **Add support for more data types.**

Extend the approach to handle a broader range of data types than just integers. Previous work [12] has explored type extensions for the current Hypra approach. Evaluate how these types can be integrated into the new syntactic approach.

- 4) **Generalize to relational properties.**

Extend the existing framework such that it can also reason about relational properties, that is, properties that relate executions of different programs.

- 5) **Support automatic framing and compositional verification.**

Evaluate how *automatic framing*, which Hypra currently supports [8], can be integrated into the new syntactic automation framework. Also, examine how *compositional verification* of distinct method calls, currently explored in a separate project [13], can be implemented with our approach.

REFERENCES

- [1] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, p. 576–580, Oct. 1969. [Online]. Available: <http://dx.doi.org/10.1145/363235.363259>
- [2] D. Volpano, C. Irvine, and G. Smith, “A sound type system for secure flow analysis,” *Journal of Computer Security*, vol. 4, no. 2–3, p. 167–187, Apr. 1996. [Online]. Available: <http://dx.doi.org/10.3233/JCS-1996-42-304>
- [3] M. R. Clarkson and F. B. Schneider, “Hyperproperties,” *Journal of Computer Security*, vol. 18, no. 6, p. 1157–1210, Sep. 2010. [Online]. Available: <http://dx.doi.org/10.3233/JCS-2009-0393>
- [4] R. Dickerson, Q. Ye, M. K. Zhang, and B. Delaware, “RHLE: Modular deductive verification of relational $\forall\exists$ properties,” in *Programming Languages and Systems*. Springer Nature Switzerland, 2022, p. 67–87. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-21037-2_4
- [5] T. Antonopoulos, E. Koskinen, T. C. Le, R. Nagasamudram, D. A. Naumann, and M. Ngo, “An algebra of alignment for relational verification,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, p. 573–603, Jan. 2023. [Online]. Available: <http://dx.doi.org/10.1145/3571213>
- [6] T. Dardinier and P. Müller, “Hyper Hoare Logic: (Dis-)proving program hyperproperties,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, p. 1485–1509, Jun. 2024. [Online]. Available: <http://dx.doi.org/10.1145/3656437>
- [7] D. McCullough, “Specifications for multi-level security and a hook-up,” in *1987 IEEE Symposium on Security and Privacy*. IEEE, Apr. 1987. [Online]. Available: <http://dx.doi.org/10.1109/SP.1987.10009>
- [8] T. Dardinier, A. Li, and P. Müller, “Hypra: A deductive program verifier for Hyper Hoare Logic,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, p. 1279–1308, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.1145/3689756>
- [9] P. Müller, M. Schwerhoff, and A. J. Summers, “Viper: A verification infrastructure for permission-based reasoning,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI)*. Springer Berlin Heidelberg, Jan. 2016, p. 41–62. [Online]. Available: http://dx.doi.org/10.1007/978-3-662-49122-5_2
- [10] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM*, vol. 18, no. 8, p. 453–457, Aug. 1975. [Online]. Available: <http://dx.doi.org/10.1145/360933.360975>
- [11] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer Berlin Heidelberg, 2008, p. 337–340. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-78800-3_24
- [12] P. Winkler, “Improving a deductive program verifier for hyperproperties,” Bachelor’s Thesis, Apr. 2025, Advisors: Anqi Li, Thibault Dardinier, and Prof. Dr. Peter Müller. [Online]. Available: https://ethz.ch/content/dam/ethz/special-interest/infk/chair-program-method/pm/documents/Education/Theses/Paul_Winkler_BS_Report.pdf
- [13] P. Neugebauer, “Automated compositional verification of hyperproperties,” Bachelor’s Thesis Project Description, Apr. 2025, Advisors: Anqi Li, Thibault Dardinier, and Prof. Dr. Peter Müller. [Online]. Available: https://ethz.ch/content/dam/ethz/special-interest/infk/chair-program-method/pm/documents/Education/Theses/Patrick_Neugebauer_BS_Description.pdf